EXPENSE TRACKER SYSTEM

Milestone 1 — ERD & Relational Database Schema

Software Engineering — Database Design Submission

Project Title	Expense Tracker System
Milestone	Milestone 1 — ERD Diagram & Relational Schema Description
Submission Type	Database Design Document
Tool Used	draw.io / MySQL Workbench

---

1. PROJECT OVERVIEW

The Expense Tracker System is a relational database application designed to help users manage their personal finances. The system enables users to:

- Record and categorise individual expenses
- Track multiple sources of income over time
- Organise expenses into user-defined categories
- Set and monitor spending budgets per category

The database is designed to support a multi-user environment where each user's data is independently managed and isolated through foreign key relationships anchored to a central users table.

2. ENTITY-RELATIONSHIP DIAGRAM (ERD)

The ERD was created using draw.io / MySQL Workbench and contains five entities. All relationships, primary keys, foreign keys, and cardinalities are shown in the diagram. The diagram follows crow's foot notation.

2.1 Entities Overview

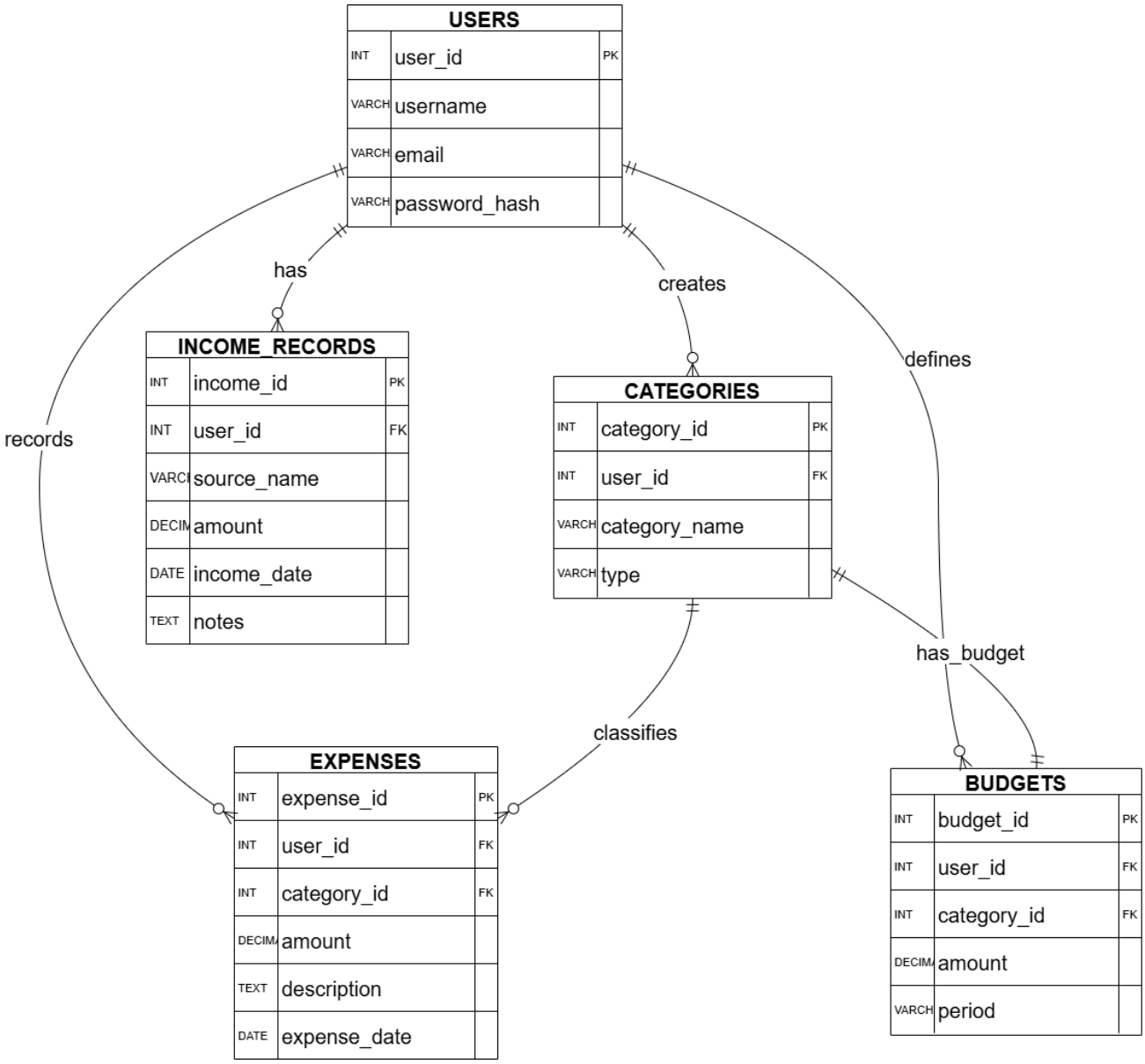
Entity	Purpose
users	Central entity. Stores user account information. All other entities trace ownership back to this table.

categories	Stores expense/income categories defined by each user. Each category belongs to one user and has a type (INCOME or EXPENSE).
expenses	Records individual expense transactions. Each expense belongs to a user and is classified under a category.
income_records	Records individual income entries for a user. Multiple income records can exist per user (salary, freelance, etc.).
budgets	Defines a spending budget limit for a specific category. Each category can have at most one budget.

2.2 Relationships and Cardinalities

From Entity	Cardinality	To Entity	Description
users	**1 : N**	**categories**	One user creates many categories. Each category belongs to exactly one user.
users	**1 : N**	**expenses**	One user owns many expense records. The direct FK ensures ownership is retained even if a category changes.
users	**1 : N**	**income_records**	One user has many income records. Income is transactional — a user may have salary, freelance, and other sources recorded separately.
users	**1 : N**	**budgets**	One user owns many budget definitions, one per category.
categories	**1 : N**	**expenses**	One category classifies many expenses. Each expense belongs to exactly one category.
categories	**1 : 1**	**budgets**	Each category may have at most one budget limit. One budget maps to exactly one category.

2.3 ERD Diagram



3. RELATIONAL DATABASE SCHEMA

The following section describes each table in the database: its attributes, data types, key constraints, and its role within the schema. All five tables are in compliance with the corrected ERD.

KEY: **PK** = Primary Key **FK** = Foreign Key **UQ** = Unique Constraint

3.1 Table: users

Stores user account information. This is the central entity of the schema. All other tables reference users through a foreign key on user_id. Using a surrogate integer primary key ensures the table identity is stable and independent of user-editable fields such as username.

users

Attribute	Key	Data Type	Constraints	Description
user_id	PK	INT	NOT NULL, AUTO_INCREMENT	Unique system-generated identifier for each user
username	UQ	VARCHAR(64)	NOT NULL, UNIQUE	Chosen display name; must be unique across all users
email	UQ	VARCHAR(255)	NOT NULL, UNIQUE	User email address; used for login identification
password_hash	—	VARCHAR(255)	NOT NULL	Hashed password (bcrypt / Argon2); never stored in plaintext

- *user_id is a surrogate key — it has no business meaning and will not change even if the username or email is updated.*
- *Both username and email carry UNIQUE constraints; either can serve as a human-readable identifier.*

3.2 Table: categories

Stores the expense and income categories created by each user. A category acts as a classification label for both expenses and budgets. Each category belongs to exactly one user and has a declared type that determines whether it classifies income or expenses.

categories

Attribute	Key	Data Type	Constraints	Description
category_id	PK	INT	NOT NULL, AUTO_INCREMENT	Unique identifier for each category
user_id	FK	INT	NOT NULL, FK → users(user_id)	Identifies the user who owns this category
category_name	—	VARCHAR(128)	NOT NULL	Human-readable name (e.g., 'Groceries', 'Salary')
type	—	VARCHAR(10)	NOT NULL, values: INCOME/EXPENSE	Classifies whether this category is for income or expense tracking

- *category_id replaces the previous unnamed/missing primary key. All FK references from expenses and budgets point to this column.*
- *The column previously named 'category' has been renamed to category_name to accurately describe its content.*
- *The 'type' field is constrained to two valid values: 'INCOME' or 'EXPENSE'. This constraint will be enforced as a CHECK constraint in the DDL (Milestone 4).*

- *The attribute last_expense_id has been removed. It was a derived value that could be calculated at query time and its presence introduced update anomalies.*
- *A UNIQUE constraint on (user_id, category_name) prevents a user from creating duplicate category names.*

3.3 Table: expenses

Records individual expense transactions. Each expense is owned directly by a user and classified under one of their categories. The direct foreign key to users ensures expense records are not orphaned if category assignments change.

expenses

Attribute	Key	Data Type	Constraints	Description
expense_id	**PK**	INT	NOT NULL, AUTO_INCREMENT	Unique identifier for each expense record
user_id	**FK**	INT	NOT NULL, FK → users(user_id)	Direct ownership link to the user who recorded this expense
category_id	**FK**	INT	NOT NULL, FK → categories(category_id)	Category under which this expense is classified
amount	—	DECIMAL(10,2)	NOT NULL, value > 0	Monetary value of the expense. DECIMAL ensures no floating-point rounding errors
description	—	TEXT	NULL allowed	Optional free-text note describing the expense
expense_date	—	DATE	NOT NULL	Date on which the expense occurred

- *expense_id is now a standalone surrogate PK. The previous composite PK (category, expense_id) has been replaced — a composite key using a FK column is fragile and unnecessary.*
- *user_id is a new addition. Previously, expenses had no direct FK to users, meaning expenses could lose their owner context if category records changed.*
- *DECIMAL(10,2) is mandatory for all monetary fields. Floating-point types (FLOAT, DOUBLE) must not be used for currency values as they cannot represent decimal fractions exactly.*
- *category_id references categories(category_id) — the corrected surrogate PK of the categories table.*

3.4 Table: income_records

Records individual income entries for a user. The relationship between users and income_records is one-to-many: a single user can have multiple income records representing different sources (e.g., salary, freelance work, rental income) and different time periods. Income is treated as transactional data, not a static property of the user.

income_records

Attribute	Key	Data Type	Constraints	Description

income_id	**PK**	INT	NOT NULL, AUTO_INCREMENT	Unique identifier for each income entry
user_id	**FK**	INT	NOT NULL, FK → users(user_id)	Identifies the user who received this income
source_name	—	VARCHAR(128)	NOT NULL	Name of the income source (e.g., 'Monthly Salary', 'Freelance Project A')
amount	—	DECIMAL(10,2)	NOT NULL, value > 0	Monetary value of the income entry
income_date	—	DATE	NOT NULL	Date on which the income was received
notes	—	TEXT	NULL allowed	Optional additional notes about the income entry

- *Relationship corrected from 1:1 to 1:N. The previous design allowed exactly one income record per user, which is factually incorrect for any real-world scenario.*
- *source_name and income_date have been added. Without these fields, individual income records cannot be meaningfully distinguished from one another.*
- *DECIMAL(10,2) is used for amount — consistent with all other monetary fields in the schema.*

3.5 Table: budgets

Defines a spending budget limit for a specific category. Each category may have at most one budget, creating a one-to-one relationship between categories and budgets. A budget is also directly owned by a user for efficient lookup.

budgets

Attribute	Key	Data Type	Constraints	Description
budget_id	**PK**	INT	NOT NULL, AUTO_INCREMENT	Unique identifier for each budget record
user_id	**FK**	INT	NOT NULL, FK → users(user_id)	Direct ownership link to the user who defined this budget
category_id	**FK,UQ**	INT	NOT NULL, UNIQUE, FK → categories(category_id)	The category this budget applies to. UNIQUE enforces one budget per category.
amount	—	DECIMAL(10,2)	NOT NULL, value > 0	The maximum spending limit for the associated category
period	—	VARCHAR(16)	NOT NULL, values: DAILY/WEEKLY/MONTHLY/ANNUAL	Time period over which the budget limit applies

- *budget_id is a new surrogate PK. The previous schema used 'category' as both PK and FK to a table that had no PK — this was structurally invalid.*

- *user_id FK added for direct user ownership. Without it, a query for 'all budgets belonging to user X' required a join through categories.*
- *UNIQUE on category_id enforces the 1:1 cardinality between categories and budgets at the database level.*
- *The period field is new. A budget amount without a time period is ambiguous — a limit of 500 means different things as a daily vs monthly limit.*
- *DECIMAL(10,2) used for amount — consistent with all other monetary fields.*

4. SCHEMA SUMMARY

4.1 Complete Attribute Reference

Table	Attribute	Data Type	Key	Constraints
users	user_id	INT	**PK**	NOT NULL, AUTO_INCREMENT
users	username	VARCHAR(64)	**UQ**	NOT NULL, UNIQUE
users	email	VARCHAR(255)	**UQ**	NOT NULL, UNIQUE
users	password_hash	VARCHAR(255)	—	NOT NULL
categories	category_id	INT	**PK**	NOT NULL, AUTO_INCREMENT
categories	user_id	INT	**FK**	NOT NULL → users
categories	category_name	VARCHAR(128)	—	NOT NULL
categories	type	VARCHAR(10)	—	NOT NULL, INCOME/EXPENSE
expenses	expense_id	INT	**PK**	NOT NULL, AUTO_INCREMENT
expenses	user_id	INT	**FK**	NOT NULL → users
expenses	category_id	INT	**FK**	NOT NULL → categories
expenses	amount	DECIMAL(10,2)	—	NOT NULL, > 0
expenses	description	TEXT	—	NULL allowed
expenses	expense_date	DATE	—	NOT NULL
income_records	income_id	INT	**PK**	NOT NULL, AUTO_INCREMENT
income_records	user_id	INT	**FK**	NOT NULL → users
income_records	source_name	VARCHAR(128)	—	NOT NULL
income_records	amount	DECIMAL(10,2)	—	NOT NULL, > 0
income_records	income_date	DATE	—	NOT NULL
income_records	notes	TEXT	—	NULL allowed

budgets	budget_id	INT	**PK**	NOT NULL, AUTO_INCREMENT
budgets	user_id	INT	**FK**	NOT NULL → users
budgets	category_id	INT	**FK, UQ**	NOT NULL, UNIQUE → categories
budgets	amount	DECIMAL(10,2)	—	NOT NULL, > 0
budgets	period	VARCHAR(16)	—	NOT NULL, DAILY/WEEKLY/MONTHLY/ANNUAL

4.2 Foreign Key Reference Map

Child Table	FK Column	Parent Table	Parent Column
categories	user_id	**users**	user_id
expenses	user_id	**users**	user_id
expenses	category_id	**categories**	category_id
income_records	user_id	**users**	user_id
budgets	user_id	**users**	user_id
budgets	category_id	**categories**	category_id

4.3 Design Decisions and Notes

The following design decisions are documented for clarity:

Surrogate Primary Keys

All five tables use an integer surrogate primary key (AUTO_INCREMENT). Natural keys such as usernames or category names are intentionally excluded from PK roles because they are mutable — a user may rename a category or change their username. Surrogate keys ensure that no FK reference ever needs to be updated when user-editable data changes.

DECIMAL(10,2) for All Monetary Values

Every column that holds a currency value uses DECIMAL(10,2). This type stores exact decimal fractions, which is critical for financial arithmetic. Floating-point types (FLOAT, DOUBLE, REAL) use binary approximations that cannot represent values like 0.10 exactly, leading to rounding errors in balance calculations, budget comparisons, and summation queries. DECIMAL(10,2) supports values up to 99,999,999.99 — sufficient for personal finance use.

Income as 1:N (not 1:1)

Income is modelled as a one-to-many relationship between users and income_records. A single user may receive income from multiple sources simultaneously (employment, freelance, investments) and may record income at irregular intervals over time. Modelling income as a single static record per user (1:1) would make it impossible to track historical income, record raises, or distinguish between income sources.

Direct user_id FK on expenses

Although expenses are linked to users through categories (expense → category → user), a direct user_id FK has been added to the expenses table. This ensures that: (a) ownership is unambiguous and queryable without a join, and (b) if a category is reassigned or restructured in a future milestone, individual expense records retain their user ownership.

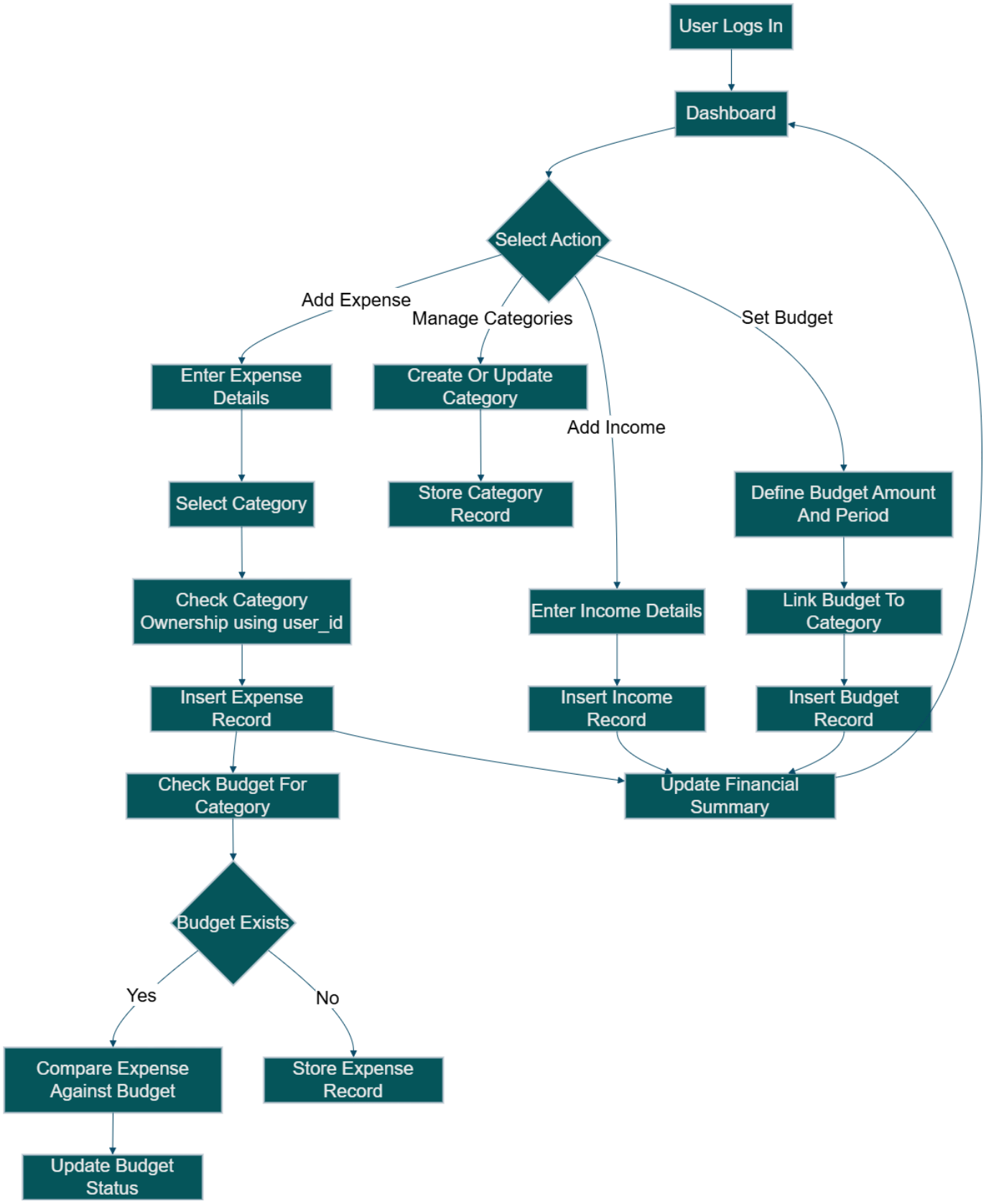
Budget Period Field

The budgets table includes a period column specifying whether the budget limit applies on a DAILY, WEEKLY, MONTHLY, or ANNUAL basis. A budget amount without a defined time period is meaningless — 500 as a daily limit is very different from 500 as an annual limit. This field makes budget evaluation logic unambiguous.

Removal of last_expense_id

The attribute last_expense_id was present in the original categories table. This value is derivable at query time (SELECT MAX(expense_id) FROM expenses WHERE category_id = X) and therefore does not belong as a stored column. Storing a derived value creates an update anomaly: it must be manually synchronised every time an expense is inserted or deleted, and it can fall out of sync silently. The column has been removed.

5. WORKFLOW DIAGRAM


User Logs In
Dashboard
Select Action
Add Expense
Manage Categories
Add Income
Set Budget
Enter Expense Details
Create Or Update Category
Select Category
Store Category Record
Define Budget Amount And Period
Check Category Ownership using user_id
Enter Income Details
Link Budget To Category
Insert Expense Record
Insert Income Record
Insert Budget Record
Check Budget For Category
Update Financial Summary
Budget Exists
Yes
No
Compare Expense Against Budget
Store Expense Record
Update Budget Status

6. ASSUMPTIONS AND SCOPE

The following assumptions apply to the Milestone 1 schema design:

- All monetary values are assumed to be in a single currency. Multi-currency support is outside the scope of Milestone 1.
- User authentication logic (sessions, tokens, password reset) is outside the scope of the database schema at this stage.
- Recurring transactions (e.g., monthly rent auto-entry) are not modelled in Milestone 1. This may be extended in future milestones.
- The schema is designed for a relational RDBMS (MySQL / PostgreSQL). No NoSQL or document-store considerations apply.
- Soft-delete and audit trail columns are not included at this stage but may be added in later milestones as requirements are formalised.